

AASHA INTERVIEW CHEAT SHEET

A Sequential Conversational Guide to React, PWA, and Offline-First Architectures

SECTION 1 — "WHAT IS AASHA?"

Tell me about AASHA.

AASHA is an offline-first healthcare application designed to help ASHA (Accredited Social Health Activist) workers capture community health information in remote areas with unreliable internet connectivity.

The Problem: In rural and low-connectivity regions, internet access is highly unstable. Health workers cannot rely on cloud-dependent applications, as dropping connections cause data loss and disrupt workflows.

The Solution: Allow health workers to capture patient records locally, continue working uninterrupted offline, and automatically synchronize the cached records to the central server when a connection returns.

Verbal Pitch (30-45 Sec Answer):

"AASHA is an offline-first community health worker assistant. In rural areas, we can't depend on continuous internet connectivity. Our solution is built to work locally first: when an ASHA worker registers a patient or logs symptoms, the React frontend extracts key structured clinical data and saves it directly to a local IndexedDB database in the browser. The worker can continue filling forms and looking up patients completely offline. Once a network connection is detected, our sync engine automatically pushes those local records to a Spring Boot API, which updates the central PostgreSQL database safely and marks the client cache as synchronized."

SECTION 2 — "WHY OFFLINE-FIRST?"

Why did you make it offline-first instead of just a standard web application?

In rural and low-connectivity environments, offline functionality is not an 'additional feature' — it is the core requirement. A standard web application expects a continuous link: **User** → **Internet** → **Backend**. If the internet fails, the application breaks immediately and the worker cannot log critical health data.

AASHA's offline-first architecture changes this by routing data locally first: **User** → **Local Storage (IndexedDB)**. The backend server and network are treated as synchronization endpoints rather than hard runtime dependencies: **Local Storage** → **Synchronize** → **Backend**.

SECTION 3 — "WHAT DOES OFFLINE-FIRST MEAN?"

What is the exact definition of offline-first, and what is its operational sequence?

Offline-first means the application is designed to operate locally first and synchronize with the server later when network is available.

Operational Flow Sequence:

User enters data ↓ Save locally (IndexedDB) ↓ Continue working offline ↓ Internet returns ↓ Synchronize records ↓ Server stores data

SECTION 4 — "WHAT IS A PWA?"

What is a PWA and how does it relate to React?

Progressive Web App (PWA) — A web application built with modern web APIs to deliver native app-like features, including desktop/mobile installation, offline support, caching, and background sync.

React vs. PWA: **React** is the UI library used to build the user interface and components. A **PWA** refers to the set of web capabilities that make that React interface work offline and run like an app. React itself does *not* make an application a PWA; PWA features must be configured separately.

React App (UI/State) + PWA Features (Manifest + Service Worker + IndexedDB) = React PWA

SECTION 5 — "HOW DOES A PWA ACTUALLY WORK?"

What are the internal mechanics of a PWA and where is the offline boundary?

A PWA utilizes a **Service Worker** to act as a local proxy. When the browser requests assets, the Service Worker intercepts the call and serves them directly from the local cache instead of requesting them over the internet.

Normal React App: React UI → Network Request → Backend API

Offline-Capable PWA: React UI → Local Cache / Local Database (Instant response)

Key Distinction: PWA does *NOT* make the backend offline. The server still requires network connectivity to receive data. The PWA simply allows the frontend to run locally on the client device using local files and databases until network is restored.

SECTION 6 — "WHAT IS A SERVICE WORKER?"

What is a Service Worker and does it store application records?

A **Service Worker (SW)** is a background browser script running on a separate thread from the main UI. It intercepts outgoing network requests, caches static resources (HTML, CSS, JS, images), and handles offline asset delivery.

UI Thread (Browser) ↔ Service Worker (Proxy) ↔ Network / Cache

No, the Service Worker does not store health records. The SW is responsible for network proxying and asset caching. Structured application records (like patient clinical logs) are stored in **IndexedDB**. Do not confuse the two: SW caches files, IndexedDB stores databases.

SECTION 7 — "WHERE DO WE STORE DATA WHEN OFFLINE?"

Where does AASHA store data when offline and how is it implemented?

If the internet is unavailable when an ASHA worker enters a record, we cannot contact the backend. We store the records locally in **IndexedDB** — a transactional, object-oriented database built into the browser.

AASHA Storage Pipeline: React Form → Health Record Object → Saved to IndexedDB → Stored Locally

AASHA Implementation: The database is managed in `src/indexeddb/db.js` using the `idb` library, defining an 'asha-health-db' with three object stores: 'records' (keys: id, index: syncStatus), 'drafts', and 'metadata'.

SECTION 8 — "WHAT HAPPENS TO ONE RECORD?"

Take me through the lifecycle of a single record, from entry to backend sync.

1. ASHA worker fills out a patient log and clicks 'Submit' in the React UI.
2. React captures form data; `patientRepository.savePatient()` performs local NLP parsing and risk scoring.
3. The repository builds a record object setting `syncStatus = 'pending'` and `updatedAt = Date.now()`.
4. The record is written to IndexedDB store using `db.js`'s `saveRecord()`.
5. The application triggers `verifyConnectivity()` to check if the network is available.
6. If the internet is offline, the sync attempt is bypassed, and the record remains safely in IndexedDB.
7. The UI updates instantly using the local IndexedDB state; the ASHA worker can continue working.
8. When internet returns, a user action (saving a new patient or loading the dashboard) triggers `syncPendingRecords()`.
9. The sync engine queries IndexedDB, fetches the 'pending' record, and POSTs it in a batch to `/api/sync`.
10. Spring Boot's `HealthRecordController` receives the batch, and validates/saves it to PostgreSQL.
11. Upon a successful HTTP 200 response, the frontend calls `markRecordSynced(id)` to set `syncStatus = 'synced'` in IndexedDB.

SECTION 9 — "HOW DOES SYNCHRONIZATION WORK?"

Explain the synchronization mechanics, endpoints, and error handling in AASHA.

Synchronization is the process of sending client-side IndexedDB records with pending statuses to the backend API.

How AASHA knows what to sync: The sync engine filters all IndexedDB records where `syncStatus` is `'pending'` or `'pending-delete'` (soft deletes).

API & Data Payload: It makes a `POST` request to `/api/sync` (batch upload) with a JSON list of records including patient metadata, risk assessments, and clinical scores. For soft-deletes, it sends a `DELETE` request to `/api/records/{id}`.

After Success: Synced records are marked as `'synced'` in IndexedDB. Deleted records are permanently removed from the local store.

Failure & Retry Handling: The sync engine calls `verifyConnectivity()`. If connectivity is down or the request fails, records are kept in IndexedDB as `'pending'`. **No background timers or automated retry loops are implemented** in `syncEngine.js`. Retries occur dynamically when a user saves a patient, refreshes the patient list, or opens the app.

Duplicate Handling: The Spring Boot backend checks if the record UUID already exists in PostgreSQL via `healthRecordService.saveFromRequest()`. If it exists, it skips database writing and returns a success response with a duplicate count. This ensures safe client retries.

SECTION 10 — "DOES THE BACKEND NEED TO BE DIFFERENT?"

Does a PWA require a specialized backend API?

No. The backend does not need to know it is serving a PWA. It continues to expose standard REST API endpoints. The complexity resides on the React client, which operates as a traffic cop. When online, it routes API calls to the server. When offline, it routes writes to IndexedDB. Later, the sync engine batches these local records and posts them. The backend's main responsibility is processing incoming payloads, performing validation, and handling duplicates safely.

SECTION 11 — "HOW DOES REACT FIT INTO ALL THIS?"

Explain the frontend architecture and key files involved.

The React frontend serves as the coordinator: displaying forms, storing drafts locally, writing clinical records to IndexedDB, and running background sync via the sync engine.

- **db.js (src/indexeddb/db.js)** — Initializes the browser database and exposes CRUD helper functions using the `idb` library.
- **patientRepository.js (src/repository/patientRepository.js)** — Data layer. Scores risk levels, writes records, deletes records offline (marks 'pending-delete'), and runs sync in background.
- **syncEngine.js (src/sync/syncEngine.js)** — Checks connectivity, compiles pending records into JSON payloads, handles batch POST uploads and DELETE requests, and updates local sync state.
- **HealthRecordController.java (backend controller)** — Exposes Spring Boot REST endpoints `/api/sync` and `/api/records/{id}` to process synchronization uploads and deletes.
- **vite.config.js (Vite configuration)** — Uses `vite-plugin-pwa` to auto-generate the service worker and cache static assets based on Workbox parameters.

SECTION 12 — "HOW IS THIS DIFFERENT FROM A NORMAL WEBSITE?"

How does AASHA contrast with a typical web application?

Normal Website: Every action requires a server trip. If the network goes down, requests fail, forms clear, and the application becomes unresponsive. The user is blocked.

AASHA Offline-First App: The application runs entirely on the client. It reads and writes to local databases immediately. Internet connectivity is used as a background sync mechanism. Workflows are never blocked by network latency or outages.

SECTION 13 — IMPORTANT DEFINITIONS

- **React:** A frontend JavaScript library used for building component-based, state-driven user interfaces.
- **PWA:** A web application that leverages modern browser features to support installability and offline usage.
- **Offline-first:** An architectural pattern where application data is written and read locally first, syncing later in the background.
- **Service Worker:** A background browser script acting as a network proxy that handles file caching and offline page loads.
- **IndexedDB:** A transactional, high-capacity client-side database used for storing structured records locally in the browser.
- **Synchronization:** The process of aligning local client data stores with the central backend database when internet is available.
- **REST API:** An HTTP-based interface used by the frontend sync engine to transmit records to the backend.

SECTION 14 — FINAL 10 QUESTIONS

1. Q: What is AASHA?

A: An offline-first healthcare app helping remote ASHA workers log health records without stable internet.

2. Q: Why did you choose offline-first?

A: Because internet is unreliable in rural areas; we must guarantee that worker progress is never lost.

3. Q: What is a PWA?

A: A web app that behaves like a native application with offline support and installability features.

4. Q: Is React itself a PWA?

A: No. React builds the interface. PWA capabilities (manifest, service worker, databases) are separate layers.

5. Q: What does a Service Worker do?

A: It proxies network traffic to cache static assets, enabling the frontend shell to load completely offline.

6. Q: What is IndexedDB?

A: A client-side transactional database built into the browser for storing structured JSON objects locally.

7. Q: Where is data stored offline?

A: Locally within browser-managed IndexedDB tables, specifically the 'records' and 'drafts' object stores.

8. Q: How does synchronization work?

A: The frontend searches IndexedDB for pending records, uploads them via batch POST to `/api/sync`, and updates local state.

9. Q: Does a PWA require a different backend?

A: No. The backend exposes standard REST APIs; the client handles offline logic and sync scheduling.

10. Q: What happens when internet comes back?

A: The next client-side database save or query triggers the sync engine to push queued records to the server.